Experiment No. 2

Topic: Agile Requirements Gathering with User Stories (For **NON-Functional Requirement**) **Aim:** To identify Nonfunctional requirement for any software development project.

Learning Outcome: Study consists of learning the following concepts

- Identify non-functional requirements for the software to be designed in each context.
- Write effective User stories for the non-functional / Quality requirements of selectedprojects.
- Acceptance criteria and Definition of done.
- Incorporating Design decisions to achieve desired quality for software systems.

Part A:

Functional vs Non-Functional Requirements

S. No	Functional	Non-Functional
1	Defines 'What' of the customer requirements	Defines 'How' of the customer requirements
2	Elicited by business analysts, specified by the customer	Defined by technical team like architects, developers etc
3	Documented as the requirements document by the business analyst	Documented as the technical document by the team
4	Defines the functionality in the system	Defines quality attribute in the system
5	Captured as a user story in agile	Captured either as a story or part of acceptance criteria of a user story
6	Customer tests and validates the functionality as specified by them	Customers cannot test these quality attributes, however, can experience while using the system
7	Customers find it easy to define these	It is hard to define by the customer

S. No	Functional	Non-Functional
8	Defines the product features	Defines the product properties

Non-Functional / Quality requirements for Software

- **Usability**. The higher the usability of software, the easier it is for users to workwith it. There are several aspects of usability, including learnability for novices,efficiency of use for experts, and handling of errors.

- **Efficiency**. The more efficient software is, the less it uses of CPU-time, memory, disk space, network bandwidth and other resources. This is important to customers in order to reduce their costs of running the software, although with today's powerful computers, CPU-time, memory and disk usage are less ofa concern than in years gone by.

- **Reliability**. Software is more reliable if it has fewer failures. Since software engineers do not deliberately plan for their software to fail, reliability depends on the number and type of mistakes they make. Designers can improve reliability by ensuring the software is easy to implement and change, by testing it thoroughly, and by ensuring that if failures occur, the system can handle them or can recover easily.

- **Maintainability**. This is the ease with which you can change the software. The more difficult it is to make a change, the lower the maintainability. Software engineers can design highly maintainable software by anticipating future changes and adding flexibility.Software that is more maintainable can result in reduced costs for both developers and customers.

- **Reusability**. A software component is reusable if it can be used in several different systems with little or no modification. High reusability can reduce the long-term costs faced by the development team

.

How to Elicit Non-Functional Requirements?

When the business analysts elicit requirements from the stakeholders on the functional aspects of the system, they also should understand from the stakeholder what attributes the system should provide in order to meet their business goals. For example, the website must be able to load within 0.5 seconds. Eliciting a non-functional requirement is more challenging than the functional requirement, as the stakeholders/users are good at articulating the functional requirements well; while they may not be aware of, or good at explicitly stating non-functional requirements. Business analysts will have to get these requirements from stakeholders by asking the right questions. NFRs are implicit by nature, meaning that people assume them to exist without being asked.

Some of the questions to be prompted that may help the business analyst to ask stakeholders how they want their system to function include the following aspects:

- **Performance –** speed of the system e.g., response time, throughput etc
- **Availability –** what is the impact if the system is not available for the customer?
- **Security –** ensuring unauthorized access to the system
- **Data Retention -** how long should the data be retained in the system for reference? This could be a regulation from the government/country.
- **Usability –** easy access for the customer/user while using the system
- **Stability –** code/system stability when dynamic changes apply due to business needs
- **Compliance –** understanding compliance needs by local laws/regulations
- **Reliability –** probability of system performing without failure
- **Recoverability –** ability of the system to recover from failure
- **Serviceability –** ease of system service when necessary
- **Data integrity –** accuracy and correctness of data maintained in the system
- **Scalability –** ability of the system to scale when more resources are added
- **Capacity –** how much the system can store information
- **Accessibility -** how easily people with the widest range of capabilities can use the system.

Non-Functional Requirements in Agile

A common challenge for an Agile team is dealing with capturing non-functional requirements in a user story. The non-functional requirements can be written as a user story and made available in the product backlog or sprint backlog. For example, "As an ecommerce website user, I want the website to be available for 99.99999%, so that I purchase products whenever I feel like"

NFR can also be included as Acceptance Criteria in a user story.

For example, for the following user story, "As an ecommerce website user, I want to search for products, so that I can purchase them". NFR as acceptance criteria would be "application responds to search request within 5 seconds from the time request is received".

Examples of non-functional requirements:

- As a customer, I want to be able to run your product on all versions of Windows fromWindows 95 on.
- As the CTO, I want the system to use our existing orders database rather than create a newone, so that we don't have one more database to maintain.
- As a user, I want the site to be available 99.999 percent of the time I try to access it, so thatI don't get frustrated and find another site to use.
- As someone who speaks a Latin-based language, I might want to run your softwaresomeday.
- As a user, I want the driving directions to be the best 90 percent of the time, and reasonable99 percent of the time.

Few More examples:

- **Performance –** funds transfer transaction should not take more than 0.5 seconds

- **Availability –** the system should be available for the users to use the system anytime of the week they need
- **Security –** payroll data is not visible to all the employees in the organization. Only the authorized personnel should have access to the data
- **Data Retention –** all healthcare data should reside in the system for 7 years. Older data can be archived and should be accessible when required
- **Usability –** the system navigation is easy for the end user and provides a seamless journey while accessing the system
- **Stability –** the code/system is stable when new changes are applied due to dynamic business needs
- **Compliance –** HIPAA compliance should be adhered when dealing with patients' data in USA
- **Reliability –** website users should be able to access the system 99% of the time without any failure
- **Recoverability –** In case of any major failures, the system should be restored within 48 hours
- **Serviceability –** the system should be serviceable immediately when there is a failure
- **Data integrity –** the system should not allow phone number to be entered in the data field
- **Capacity –** the system should be able to store 1 million records of basic customer information
- **Scalability –** the system should be scalable to support unlimited growth of the employees
- **Accessibility -** The system shall be accessible to people with disabilities in accordance with the Americans with Disabilities Act
- **Confidentiality –** The system shall not display the bank account number in full. It should display only the last 4 digits of the account number while others can be masked
- **Efficiency –** The interface between the system and user should not take more than 2 seconds
- **Portability –** The system shall be developed for both windows and Mac operating systems platforms

For performance requirements and many other nonfunctional requirements(NFR's), one can use constraints and stories.

- Usually create a story to document the NFR and
- Define story tests for it.
- Then, add the story tests as a "constraint."

A constraint is something that all implemented stories(features and functionality) must comply with.

SAMPLE:

Step 1: Identify and quantify the constraint, put it in terms that your users and business stakeholders will understand, and put it into one or more stories as Story Tests (akaAcceptance Tests).
Story Title: System response time

- **Story Test #1:** Test that the system responds to all non search requests within 1 second ofreceiving the request
- **Story Test #2:** Test that the system responds to all search requests within 10 seconds ofreceiving the request

Some things to keep in mind:

- If you cannot quantify the story in concrete terms, this should be a bad smell that usually indicates a requirement that is too vague to be implemented. Vague NRF's have the sameproblems that vague functional requirements do: It is hard to answer the question "How will I know when this story is correctly done?"
- Be sure not to specify a technical solution or implementation in the story, because stories are about **"The What"**("**What** the user wants") and they are not about **"The How"** ("**How** this is implemented").
- Plan, estimate, split(if necessary), and implement this story like all other user stories, aspart of the Product Backlog(Scrum).

Once this story is complete, the entire system will be in compliance with this particularconstraint.

If your constraint is not system-wide or far reaching:

- Just add it as a story test for that story. But again, specify the requirement, not theimplementation, in terms the business stakeholders will understand.

The decision to create a constraint or not will rest on whether the constraint should be considered in at least several future stories(or system wide).
If it will apply to several future stories, then create a constraint.
If it won't apply to several future stories, then just add the NFR as a storytest to the stories that it applies to or create a separate story to comply with the NFR in the smallpart of the system that requires it.

Step 2: Add the NFR related Story Tests to your list of constraints (and to your Definition of Done if you're doing Scrum)
Publish your list of constraints (and/or DoD) somewhere that is highly visible. Even if you keep your constraints electronically, print them out in large print and post them somewhere on your Scrum board or in your team area.

Constraints
• **Test that the system responds to all non search requests within 1 second of receiving the request.**
• **Test that the system responds to all search requests within 10 seconds of receiving the request.**
• Test that the system logs a user out after 10 seconds of inactivity and redirects their browser to the home page.
• Test that any update to a person's payment information(anywhere in the system) is loggedto the payment_preferences log, along with the following information: ○ IP Address of logged in person   ○ Old preference value, new preference value   ○ Date/time of change
• Test that any time a person's credit card number is shown in the application, that only the last 4 digits display.

Add the constraints to your Definition of Done.

Definition of Done
• **All stories must comply with all of the story constraints<link to constraints page on wiki>.**
• All code must be peer reviewed within 4 hours of checkin.
• If a change is made to the web services interface, the change must be documented on the official web services api wiki page<link to api on wiki>.
• All code must have automated testing that is consistent with the "Automated Testing Guidelines"<link to guidelines on wiki>
• Any change in of functionality that is visible in the GUI must be at least tested manually(automated tests also acceptable) against the integration environment before making the functionality available for a QA review.

Part B:

- Create Document of NFR for your selected Software application.

Name	Eklavya Vail	Aarush Mishra	Mithas Jain
Roll no.	B117	B155	B160

Course: Btech
Branch: CE
Division: C2

Non-Functional Requirements (NFR) Document

Patient Care Platform

Methodology: Agile Requirement Gathering

1. Quality Attribute Explanations

Non-functional requirements define the properties and quality attributes of a system rather than its specific features. For Patient Care Platform, these attributes ensure the platform is trustworthy, smooth, and dependable for vulnerable users.

- **Security & Confidentiality:** In healthcare, privacy is paramount. This attribute ensures that unauthorized personnel cannot view a patient's medical history or daily vitals. It also ensures that the menstrual data of the women is not visible to others (even the doctors) without their consent.

- **Performance & Efficiency:** This dictates the speed and responsiveness of the system. When a caregiver is checking on a patient during a potential emergency, the system must not waste time processing data and it should give the updates to the doctor timely.

- **Availability & Reliability:** Health monitoring never sleeps. This attribute ensures the platform is online and functioning correctly around the clock, so critical alerts are never missed due to a system crash.

- **Usability & Accessibility:** The system must be incredibly easy to navigate. Since many patients using the platform may be elderly or unfamiliar with complex software, the design must prioritize simple, intuitive interactions.

- **Maintainability:** Code should be well defined and easy to update, so that new features can be added rapidly in case of any emergency like a pandemic, with minimal disruption to existing services.

2. NFR User Stories & Story Tests

By writing non-functional requirements as user stories, we can focus on what the user actually wants to experience, avoiding overly technical implementation details in the planning phase.

Story 1: Data Confidentiality (Security)

- **User Story:** As a Patient, I want my sensitive medical data and identity details masked on shared screens, so that my personal health information remains confidential if someone happens to glance at my device.

- **Story Test 1:** Test that any display of the patient's personal identification numbers shows only the last few digits, masking the rest of the information.

- **Story Test 2:** Test that all uploaded medical documents (like lab reports) are passed through standard encryption before being saved to the cloud database, ensuring they cannot be read if intercepted.

Story 2: System Performance (Efficiency)

- **User Story:** As a Caretaker, I want the patient health dashboard to load smoothly and quickly, so that I don't feel frustrated or delayed when checking on my loved one's vital trends during a stressful moment.

- **Story Test 1:** Test that the system responds to basic actions like submitting a daily diet log or clicking to a new page - almost instantly, providing a seamless journey for the user.

- **Story Test 2:** Test that heavier analytical requests, such as generating a 6-month historical chart of blood pressure trends, load efficiently without freezing the application.

Story 3: High Availability (Reliability)

- **User Story:** As a Patient's Family member, I want the system to be highly reliable and always online, so that I have peace of mind knowing I will never miss a critical health alert due to unexpected downtime.

- **Story Test 1:** Test that the system architecture supports high availability, meaning users can access the platform at any time of the week they need.

- **Story Test 2:** Test that emergency anomaly notifications are triggered and sent to the caregiver's device immediately after a dangerous vital reading is recorded.

Story 4: Accessibility (Usability)

- **User Story:** As an Elderly Patient, I want the interface to be visually clear with straightforward navigation, so that I can easily log my daily vitals without needing to call my family for technical assistance.

- **Story Test 1:** Test that the application design includes accessibility options, such as high-contrast colors and large text, making it readable for those with vision impairments.

- **Story Test 2:** Test that the process to log daily health data requires minimal clicks, keeping the system interface highly efficient and easy to learn.

3. Global System Constraints

A constraint is a requirement that all implemented stories, features, and functionalities must comply with system-wide.

- **Latency Constraint:** Test that the system responds to all general, non-search requests within a fraction of a second of receiving the request to maintain a smooth user experience.

- **Session Security Constraint:** Test that the system logs a user out automatically after a set period of inactivity and redirects their browser to the home page, preventing unauthorized access if a device is left unattended.

- **Audit Trail Constraint:** Test that any critical update to a user's permissions or medication schedule is safely logged in the background. This log must capture who made the change, the date and time, and what exactly was changed.

- **Data Retention Constraint:** Test that all healthcare data securely resides in the system for several years to maintain a long-term medical history, keeping in line with standard healthcare retention practices.

4. Definition of Done

To ensure high quality, the team must satisfy this checklist before any feature is considered officially complete and ready for the user.

- **Constraint Compliance:** All completed stories must comply with all of the established global system constraints.

- **Peer Review:** All newly written code must be reviewed by another team member shortly after it is committed. Additionally, the implemented functionality should be verified by the volunteer doctors and caretakers to ensure it meets the intended requirements.

- **Usability Verification:** Any change of functionality that is visible on the screen must be manually tested for accessibility and ease of use before being handed over for a final Quality Assurance review.